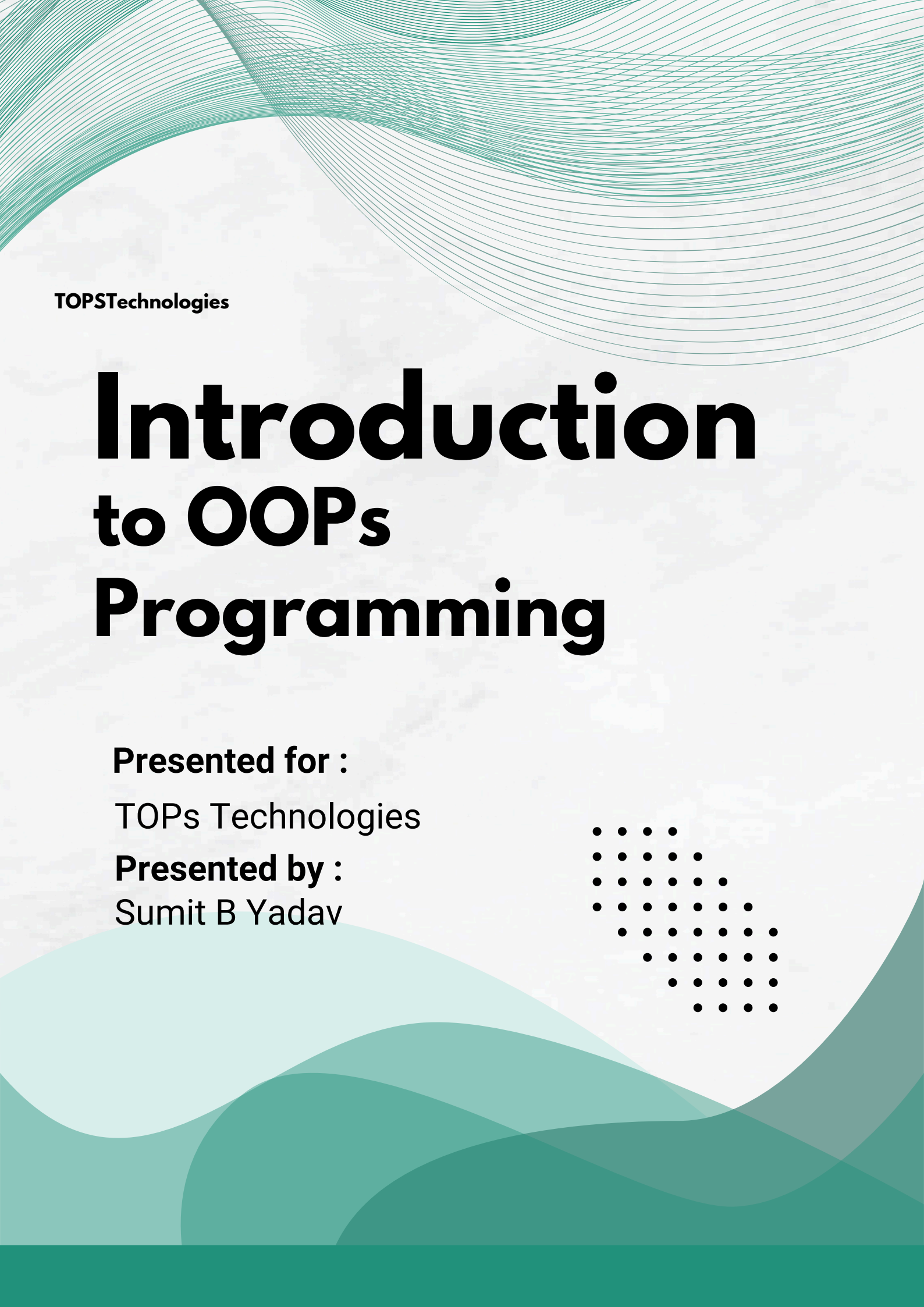




TOPSTechnologies

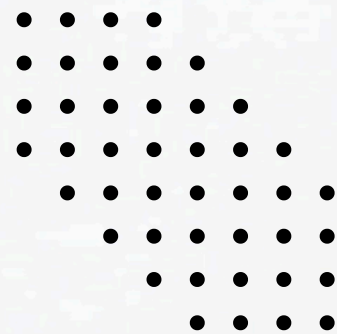
Introduction to OOPs Programming

Presented for :

TOPs Technologies

Presented by :

Sumit B Yadav



Basic Structure of a C++ Program

Que 1

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects rather than functions or logic. In OOP, objects are instances of classes, which define the blueprint for creating those objects. This approach helps in organizing and structuring code, making it more modular, reusable, and maintainable.

The four key principles of OOP are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

1. Encapsulation

Encapsulation refers to the bundling of data (variables) and methods (functions) that operate on the data into a single unit known as a class. It also involves restricting access to some of the object's components to protect its internal state. This is typically done using access modifiers (such as public, private, and protected) to control how the data and methods are accessed outside the class.

2. Abstraction

Abstraction is the concept of hiding the complex implementation details of a system and exposing only the essential features to the user. This helps in reducing complexity and allows the user to interact with an object in a simplified manner.

Abstraction can be achieved through:

- **Abstract classes:** A class that cannot be instantiated directly and may contain pure virtual functions (functions without implementation).
- **Interfaces:** A form of abstraction that specifies a contract for classes to follow.

Basic Structure of a C++ Program

3. Inheritance

Inheritance allows one class (the child class or derived class) to inherit the properties and methods of another class (the parent class or base class). This allows the child class to reuse code from the parent class and can also add or modify functionality as needed.

Key Points:

- *Base class: The class that provides common functionality.*
- *Derived class: The class that inherits from the base class.*
- *Method Overriding: Derived classes can override base class methods to provide their own specific behavior.*

4. Polymorphism

Polymorphism allows you to use a single interface to represent different underlying forms (data types). The most common types of polymorphism in OOP are:

- *Compile-time polymorphism (also known as static polymorphism), achieved through function overloading and operator overloading.*
- *Runtime polymorphism, achieved through method overriding (using virtual functions).*

Key Points:

- *Method Overloading: Defining multiple functions with the same name but different parameters.*
- *Method Overriding: A derived class provides its own implementation of a method that is already defined in the base class.*

Basic Structure of a C++ Program

Que 2

Class in C++

A class is a user-defined data type that encapsulates data and functions (called methods) which operate on that data. A class acts as a blueprint for creating objects. It defines the properties (data members) and behaviors (member functions) that objects of that class will have.

Key Components of a Class:

- Data Members: These are the variables that hold the data of the class. They represent the state of an object.*
- Member Functions: These are the functions that define the behaviors or actions that can be performed on the data members. They typically operate on the class's data members.*

Object in C++

An object is an instance of a class. When you create an object of a class, you are creating a specific instance that holds its own copy of the data members defined in the class. Each object can have different values for the same data members.

An object is created by using the class name as a type, followed by the object name.

Que 3

Inheritance is one of the core principles of Object-Oriented Programming (OOP). In C++, inheritance allows a class to derive properties and behaviors (data members and member functions) from an existing class. The class that is being inherited from is called the base class (or parent class), and the class that inherits is called the derived class (or child class).

Key Points of Inheritance:

- **Base Class:** The class that provides the properties and methods to be inherited.
- **Derived Class:** The class that inherits the properties and methods of the base class and can also add or modify its own properties and methods.
- **Access Specifiers:** The visibility of base class members in the derived class depends on the inheritance type (public, private, protected).

Types of Inheritance in C++:

1. **Single Inheritance:** A derived class inherits from a single base class.
2. **Multiple Inheritance:** A derived class inherits from more than one base class.
3. **Multilevel Inheritance:** A class is derived from another class, which is itself derived from another class.
4. **Hierarchical Inheritance:** Multiple derived classes inherit from a single base class.
5. **Hybrid Inheritance:** A combination of two or more types of inheritance.

Basic Structure of a C++ Program

Types of Inheritance in C++:

1. *Single Inheritance: A derived class inherits from a single base class.*
2. *Multiple Inheritance: A derived class inherits from more than one base class.*
3. *Multilevel Inheritance: A class is derived from another class, which is itself derived from another class.*
4. *Hierarchical Inheritance: Multiple derived classes inherit from a single base class.*
5. *Hybrid Inheritance: A combination of two or more types of inheritance.*

Basic Structure of a C++ Program

Example of Inheritance in C++

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
    // Public data members
    string name;
    int age;

    // Constructor
    Animal(string n, int a) : name(n), age(a) {}

    // Member function
    speak() {
        cout << "Animal makes a sound" << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    // Additional data member for the derived class
    string breed;

    // Constructor for derived class
    Dog(string n, int a, string b) : Animal(n, a), breed(b) {}

    // Overriding the speak function
    speak() {
        cout << "Dog barks" << endl;
    }

    // New function for the derived class
    void displayInfo() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Breed: " << breed << endl;
    }
};
```

Basic Structure of a C++ Program

```
main() {  
    // Creating an object of the derived class  
    Dog dog1("Buddy", 3, "Golden Retriever");  
  
    // Accessing methods and properties from both the base and derived classes  
    dog1.speak();           // Calls the overridden method in Dog class  
    dog1.displayInfo();     // Displays the info using derived class method  
  
    // Creating an object of the base class  
    Animal animal1("Elephant", 10);  
    animal1.speak();        // Calls the method from the base class  
}
```

Basic Structure of a C++ Program

Que 4

Encapsulation is one of the four fundamental principles of Object-Oriented Programming (OOP). It refers to the bundling of data (attributes) and the methods (functions) that operate on that data into a single unit, called a class. Encapsulation helps restrict direct access to some of the object's components, which protects the object's internal state and ensures that it is only modified in well-defined ways. In simpler terms, encapsulation allows you to hide the internal details of an object and only expose the necessary parts to the outside world, controlling how the object's data can be accessed and modified.

Encapsulation in C++ is achieved through classes and access specifiers.

1. **Class:** A class contains the data members and member functions. The data members are usually declared private to prevent direct access from outside the class.
2. **Access Specifiers:** These are used to define the accessibility of the members of a class.
 - **Private:** Members declared as private are accessible only within the class itself and not from outside. This ensures data hiding.
 - **Public:** Members declared as public are accessible from outside the class. This typically includes methods that provide controlled access to the private data.
 - **Protected:** Members declared as protected are accessible within the class and by derived classes (in case of inheritance).